

requirement $\rightarrow$ efficient and accurate on-device inference

Quantization scheme that allows inference to be carried out using integer only arithmetic, which can be implemented more efficiently than floating point inference on commonly available integer-only hardware

Training procedure to preserve end-end model accuracy post quantization

Successful deployment of CNN's requires small model sizes (limited device memory and low latency to maintain user engagement)

To reduce model size & inference time of CNN's with minimal accuracy losses

First approach $\rightarrow$ design novel architectures

Second approach $\rightarrow$ Quantize the weights/activations from 32 bit Floating point into lower bit depth representations

* Quantization Scheme $\rightarrow$ quantizes both wts and activations as 8 bit integers and just a few params (bias vectors) as 32 bit

* Quantized inference framework $\rightarrow$ Efficiently implementable on integer arithmetic only hardware

* Quantized training framework $\rightarrow$ to minimize the loss of accuracy from quantization on real models

Quantization Approaches lacked

- (i) A proper baseline architecture to be evaluated on
- (ii) Many approaches did not deliver verifiable efficiency improvements on real hardware

Quantization Replacing a high-precision numerical representation with a low precision numerical representation

might include $\rightarrow$ weights, activations

Objective $\rightarrow$ to reduce costs such as Memory, latency, computation (while preserving accuracy)

Quantized Inference

quantization scheme (integer only) $q \rightarrow$ quantized values
 $r \rightarrow$ real values

integer only arithmetic during inference

floating point arithmetic during training

affine mapping A geometric operation that preserves straight lines, parallel lines and the ratio of distances between points

its composed of a linear transformation followed by translation

Quantization scheme, an affine mapping of integers 'q' to real numbers 'r'

$$r = S(q - Z) \quad S, Z \text{ constants are quantization parameters}$$

$y = S(q - z)$ s.t. constants are quantization parameters

Single set of quantization parameters for all values within each activation > and weights array.
Separate arrays use separate quantization parameters

for B-bit quantization q is quantized as B-bit integer

constant S ("for scale") $\rightarrow$ arbitrary positive real number

typically represented as floating point quantity

constant z (for "zero-point") is the same type as quantized values

$\hookrightarrow$ is offset the quantized value q corresponding to the real value 0

quantized buffer

template <typename QType> // eg QType = int8

struct QuantizedBuffer {

vector<QType> q // the quantized values

float s // the scale

QType z // the zero point

Integer Arithmetic only matrix Multiplication

inference using integer only arithmetic?

quantized values computation involving only integer arithmetic even though scale values S are not integers

two square $N \times N$ matrices of real numbers x_1 and x_2

$$x_3 = x_1 x_2$$

entries of matrices x_α ($\alpha = 1, 2, 3$) as $x_\alpha^{(i,j)}$ $1 \leq i, j \leq N$

quantization parameters as $s_\alpha z_\alpha$

quantized entries as $q_\alpha^{(i,j)}$

$$\Rightarrow x_\alpha^{(i,j)} = s_\alpha (q_\alpha^{(i,j)} - z_\alpha)$$

$$s_3 (q_3^{(i,j)} - z_3) = \sum_{j=1}^N s_1 (q_1^{(i,j)} - z_1) s_2 (q_2^{(j,j)} - z_2)$$

$$\hookrightarrow q_3^{(i,j)} - z_3 = \sum_{j=1}^N (q_1^{(i,j)} - z_1) (q_2^{(j,j)} - z_2) \quad \text{Multiplier } M = \underline{s_1 s_2}$$

$$q_3^{(i,j)} = z_3 + M \sum_{j=1}^N (q_1^{(i,j)} - z_1)(q_2^{(i,j)} - z_2) \quad \text{Multiplier } M = \frac{S_1 S_2}{S_3} \quad (\text{only non-integers})$$

from empirical calculations, they found it to be always in the interval $(0,1)$

so they express it in a normalized form $M = 2^{-n} M_0$ M_0 is in interval $[0.5, 1)$
 n is non-negative integer

Zero point

They wanted some integer $q_i = z_i$ to correspond exactly to $x=0$

purpose unit making zero representable

↳ it is for allowing the real value zero to align with an arbitrary quantized integer value

Why do they care about exact zero?

Zero padding around boundaries

Many neural network operators introduce literal zeros

the authors wanted $x=0$ to be represented exactly in the quantized domain

$$M = 2^{-n} M_0 \quad 0.5 \leq M_0 < 1$$

they are taking a positive real number M and rewrite it as $M = (\text{power of two}) \times (\text{normalized number})$

What is bit shift?

Suppose 1000_2 which equals 8 in decimal
 (binary)

Right shift by one position by 2 positions

100_2 equals 4 10_2 equals 2

$x \gg n$ approximately performs $x/2^n$ for integers

Thus 2^{-n} can be implemented using right shift

Efficient handling of Zero-Points

(to not perform $2N^3$ subtractions and without having to expand the operands of the multiplications into 16-bit integers)

$$\text{eq. (4)} \quad q_3^{(i,k)} = z_3 + M \sum_{j=1}^N S_1(q_1^{(i,j)} - z_1) S_2(q_2^{(j,k)} - z_2)$$

$$q_3^{(i,k)} = z_3 + M \left(N z_1 z_2 - z_1 a_2^{(i)} - z_2 a_1^{(i)} + \sum_{j=1}^N q_1^{(i,j)} q_2^{(j,k)} \right)$$

$$a_2^{(i)} = \sum_{j=1}^N q_2^{(j,k)}, \quad a_1^{(i)} = \sum_{j=1}^N q_1^{(i,j)}$$

$$a_2^{(k)} = \sum_{j=1}^N q_{1,j}^{(1,k)}, \quad a_1^{(-1)} = \sum_{j=1}^N q_{1,j}^{(1,1)}$$

take only N additions to compute, so collectively they take only $2N$ additions

Rest of the cost is concentrated in one integer matrix multiplication accumulation

$$\sum_{j=1}^N q_{1,j}^{(1,1)} q_{2,j}^{(1,k)} \quad \text{which takes } 2N^3 \text{ arithmetic operations}$$

The factored out computation of $a_2^{(k)}$ and $a_1^{(-1)}$ enable hw-overhead handling of arbitrary zero-points for anything but the smallest values of N

Implementation of a typical fixed layer

→ they define data types of all quantities involved, modify quantized matrix multiplication to merge the bias-addition and activation function evaluation directly into it

$$\sum_{j=1}^N q_{1,j}^{(1,k)} q_{2,j}^{(1,k)} \rightarrow \text{int } 32 += \text{uint } 8 * \text{uint } 8$$

accumulator = accumulator + product

in-order to have the quantized bias-addition be the addition of an int 32 bias into this int 32 accumulator

the bias vector is quantized such that it uses int 32 as its quantized data type

it uses 0 as its quantization zero-point Z_{bias} , and its quantization scale S_{bias}

$$S_{bias} = S_1 S_2 \quad Z_{bias} = 0$$

after getting final value of the int 32 accumulator

(i) Scale down to the final scale used by the 8-bit output activations

(ii) Cast down to uint 8 and apply the activation function to yield the final 8-bit output activation

2.3 deals with having to compute $(z_1 - z_1) \times (q_2 - z_2)$ for every multiplication

(Can they avoid doing all these zero point subtractions repeatedly)

$$2N^3 \rightarrow (N \times N \times N)$$

↓ ↓ ↓

2 subtractions multiply terms each output entry requires N products

after defining a_1 & a_2

$N \rightarrow$ rows

so $2N$ quantities

after defining a_1 & a_2

$N \rightarrow$ rows
 $N \rightarrow$ columns

So $2N$ quantities

each requires N additions

So $2N^2$

they rearrange for making Zero-point corrections cheaper

and for most computation to remain the standard matrix multiplication

Why fusing?

Suppose Matrix multiply
↓
Add Bias
↓
ReLU

three steps
normally

So they combine it into one pipeline

uint8 $\rightarrow$ unsigned 8-bit integers

Range $0 \rightarrow 255$

because $2^8 = 256$ possible values

int32 Signed 32-bit integers

Range roughly -2^{31} to $2^{31}-1$

Why not accumulate into uint8

suppose $255 \times 255 = 65025$

already larger than uint8

Now imagine summing hundreds of such products

Result becomes much larger,

so the intermediate sum needs a large container

Accumulator $\Rightarrow$?? eg $\text{sum} = 0$
for

When paper says int32 accumulator

it means the running matrix
multiplication sum is stored
in a signed 32 bit integer

$\text{sum} += \text{product}$

The variable sum is accumulator

Why Bias int32?

Bias errors are dangerous because

$\rightarrow$ one bias is reused many times

$\rightarrow$ quantization error would repeatedly appear

$S_{\text{bias}} = S_1 S_2$ (the authors want bias values to live in the same numerical scale as the accumulator)

After accumulation
→ Rescale
→ Convert to int8
→ Apply activation

Saturating Cast ??

suppose ints can store only 0 to 255
what if value is 300

The paper observes that some activations behave like simple bounds/clamps

A normal math conversion would overflow

A saturating cast instead does $300 \rightarrow 255$

Therefore parts of activation behavior can effectively be absorbed into final clipping step

Similarly, $-20 \rightarrow 0$

So $\left. \begin{array}{l} \text{if } x > 255 \rightarrow 255 \\ x < 0 \rightarrow 0 \\ \text{otherwise } x \end{array} \right\}$ this is saturation

3 Training with simulated Quantization

Common Approach to training quantized networks



Train in floating point → then quantize the resulting weights

(sometimes with additional post-quantization training for fine-tuning)



Approach works significantly well for large models with considerable representational capacity but leads to significant accuracy drops for small models

Common failure modes for post training quantization include

(1) large differences (more than 100x) in ranges of weights for different output channels

(all channels of same layer be quantized to the same resolution, which causes weights in channels with smaller ranges to have much higher relative error)

(2) Outlier weight values that make all remaining weights less precise after quantization

Forward pass → simulates quantization effects of training

Backpropagation → still happens as usual

weights and biases are still stored in floating point

Forward propagation simulates quantized inference



→ weights are quantized before they are combined with the input

1 1 +1 ... 1 1 1 ... 1 1 +1 normalization operation ... "add back int"

→ Weights are quantized before they are convolved with the input

if batch norm is used for the layer the batch normalization parameters are "folded into" the weights before quantization

→ Activations are quantized at points where they would be during inference

For each layer, quantization is parametrized by the no of quantization levels and clamping range and is performed

force 's' into the interval $[a, b]$ by applying point wise the quantization function q defined as

$$\text{clamp}(x, a, b) = \min(\max(x, a), b)$$

(Scale) Spacing between neighbouring quantisation levels

$$S(a, b, n) = \frac{b-a}{n-1}$$

$$g(r, a, b, n) = \left[\frac{\text{clamp}(r, a, b) - a}{s(a, b, n)} \right] s(a, b, n) + a$$

The interval gets divided into evenly spaced buckets

- 1 Clamp
- 2 divide by step size
- 3 Round
- 4 Multiply back

$x \rightarrow$ real value number to be quantized

$[a, b] \rightarrow$ quantization range

$n \rightarrow$ no of quantization levels

L7 → denotes rounding to nearest integer

The equation is moving the real value into the nearest quantization level

Learning Quantization ranges

treated differently for wt quantization vs activation quantization

→ for weights $\text{set } a = \min w$
 $\text{set } b = \max w$

apply minor tweaks so that weights, once quantized as int8 values only range in $[-127, 127]$ and never take value -128

→ for activations, ranges depend on the inputs to the network

to estimate ranges, we collect $[a, b]$ ranges seen on activations during training, and then aggregate them via exponential moving averages with the smoothing parameter close to 1

Algorithm Quantized graph training and inference

1) Create a training graph of the floating point model

ii) Insert fake quantization TensorFlow operations in locations where tensors will be downcasted to fewer bits during inference

to fewer bits during inference

- iii) Train in simulated quantized mode until convergence
- iv) Create and optimize the inference graph for running in a low bit inference engine
- v) Run inference using the quantized inference graph

Batch Normalization folding

Additional complexity here The training graph contains batch normalization as a separate block of operations

Whereas the inference graph has batch norm params "folded" into convolutional or fully

EMA is used to estimate a

stable activation range

goal is to prevent quantization parameters from changing too violently between training steps

connected layers weights and biases, for efficiency

$$w_{fold} = \frac{y_n}{\sqrt{EMA(\sigma_B^2) + \epsilon}}$$

↖ batch norm scale parameter

- * Simulate quantization during the forward pass while keeping optimization in floating point
- * Forward pass simulates quantized inference while parameters remain stored in floating point
- * Quantization of weights and activations are handled differently

Why do Outliers make the remaining weights less precise?

Suppose wts are [1, 1]
except one is 100

quantization range must cover [1, 100]

So → same no of quantization levels, but much larger interval
(so distance bwn neighboring quantized values becomes larger)

Result

Most wts now receive coarser representation

Batch-Norm Folding

Before folding

Conv
↓
BatchNorm
↓
Output

Inference time

Batch norm parameters are fixed

No longer changing

So Batch norm effect can be absorbed into

• convolution wts & biases

creating

Modified Conv

Modified Conv
↓
Output

Experiments summary

Trying to prove Integer only quantization maintains accuracy and improves latency

Quantization not destroying model quality, Accuracy loss is small → ResNet experiments

This method generalizes beyond ResNets

MobileNet experiments → better latency
Accuracy remains very close
Latency accuracy tradeoff improves

Method works beyond classification tasks

Ablation studies

Tried to determine sensitivity to weight precision, activation precision

Summary

- 1 Integer only inference can closely approximate floating point inference
- 2 Simulated quantization training is necessary to preserve accuracy
- 3 The resulting models achieve a better latency-accuracy tradeoff on the mobile hardware
- 4 Quantization and efficient architecture design complement each other